LECTURE 1

Application Security

This lecture introduces application security as the discipline of protecting software throughout its life cycle. It begins by positioning application security within the wider field of cybersecurity and explaining the shift-left principle, and then presents the core security objectives, the environments in which applications run, and the models, frameworks, and design principles that structure an application security program. Threat modeling is then developed in depth, covering data flow diagrams, trust boundaries, STRIDE, and risk ranking, and is applied in a worked example. The chapter closes with a set of secure coding practices.

IN THIS LECTURE

1.1 Overview of Application Security
1.2 Security Objectives of an Application
1.3 Application Security Administration
1.4 Application Security Models
1.5 Security Frameworks and Standards
1.6 Threat Modeling
1.7 Worked Example: QuickBite Threat Model
1.8 Secure Coding Practices

1.1 Overview of Application Security

Cybersecurity is the broad discipline that protects digital systems and information. It encompasses network security, infrastructure security, and physical security, among other areas. **Application security is a specific subset of cybersecurity, focused exclusively on securing software.** This specialized focus involves identifying and mitigating risks that could allow unauthorized access, data theft, or system disruption.

Application security is the set of measures and practices designed to protect software applications from threats and vulnerabilities throughout their life cycle. It aims to ensure that applications operate securely, safeguarding their integrity, confidentiality, and availability.

Although the security domains are interconnected and may share certain controls and practices, each addresses a different layer of the broader technology environment. Establishing the position of application security within this landscape, and distinguishing it from the other domains, provides an important foundation for understanding the secure coding practices discussed later in this chapter.

Table 1.1 compares the four domains according to the layer they protect and the primary mechanisms commonly used to secure it. As the comparison shows, application security has a distinct focus: it protects the software itself, including its architecture, source code, third-party

dependencies, and behavior during execution. This differentiates it from security disciplines that primarily address the surrounding network, computing infrastructure, or physical environment.

Table 1.1 *Domains of cybersecurity and their focus areas*

Area	What it protects	Typical measures
Network security	Traffic and connections between systems	Firewalls, segmentation, intrusion detection, VPNs
Infrastructure / cloud security	Servers, containers, virtual machines, cloud accounts	Hardening, host patching, identity and access management, configuration baselines
Physical security	Buildings, data centers, devices	Access badges, locks, cameras
Application security	The software itself: its design, code, dependencies, and runtime behavior	Threat modeling, secure coding, code review, SAST/DAST, dependency scanning, application monitoring

1.2 Security Objectives of an Application

Application security is fundamentally concerned with protecting information and functionality. Three classical security objectives provide the foundation for this protection: confidentiality, integrity, and availability, commonly abbreviated as the **CIA triad**.

1.2.1 Confidentiality

Confidentiality means that data is accessible only to authorized parties. When confidentiality is breached, unauthorized individuals can read data they should not see.

> EXAMPLE
> A healthcare application stores patient records. If an attacker can access those records without authorization, confidentiality is breached. This is not just a technical failure; it is a legal violation under regulations such as HIPAA.

Typical failures: missing access controls, unencrypted data at rest or in transit, verbose error messages that leak information, and insecure direct object references.

1.2.2 Integrity

Integrity means that data cannot be modified by unauthorized parties. When integrity is breached, data becomes unreliable, because it may have been altered without detection.

> EXAMPLE
> A banking application allows funds transfers. If an attacker can modify the amount of a transfer after it is submitted, integrity is breached and the bank's records no longer reflect reality.

Typical failures: missing input validation, lack of digital signatures, insufficient audit trails, and SQL injection that modifies database records.

1.2.3 Availability

Availability means that the application and its data are accessible when legitimate users need them. When availability is breached, the application becomes unusable.

> EXAMPLE
> An e-commerce platform is operating during a holiday sale. If an attacker floods the site with requests and takes it offline, availability is breached. The business loses revenue, and customers lose trust.

Typical failures: denial-of-service attacks, resource exhaustion, unpatched systems that crash under load, and inadequate infrastructure capacity.

1.2.4 Authentication, Authorization, and Accountability

Although the CIA triad provides the fundamental security objectives, modern applications also require mechanisms for authentication, authorization, and accountability. **Authentication** determines who a user, service, or system is. **Authorization** determines what an authenticated identity is permitted to do. **Accountability** provides evidence of actions performed within the system, supported by audit trails, logging, timestamps, and appropriate protection of security records.

1.2.4.1 Authentication

Authentication means verifying that a user, device, or system actually is who it claims to be, before any data or functionality is made available. When authentication is weak, an attacker can step into a legitimate identity without ever needing to break confidentiality or integrity controls directly.

> EXAMPLE
> A hospital's electronic health record (EHR) system requires clinicians to log in with a smart card and a PIN before viewing any patient data. If an attacker guesses a weak password or reuses a credential stolen from another, unrelated breach, the system may authenticate the impostor as if they were the legitimate clinician. Authentication failures do not disclose data by themselves; they simply open the door that authorization and encryption controls must then still hold.

Typical failures: weak or reused passwords, no multi-factor authentication, predictable or long-lived session tokens, and insecure credential storage.

1.2.4.2 Authorization

Authorization means determining what an already-authenticated identity is actually permitted to do or see. Authentication and authorization are frequently confused, but a system can

authenticate someone correctly and still fail catastrophically if it does not then check what that person is allowed to access.

> **EXAMPLE**
> In a multi-tenant SaaS billing platform, a logged-in customer-support agent should be able to view invoices only for the customers assigned to them, not the invoices of every company on the platform. If the application checks merely that the requester is authenticated, rather than that they are authorized for that specific tenant's data, any support agent could view every customer's invoices simply by changing an account ID in the URL or API request.

Typical failures: missing object-level authorization checks, relying on hidden buttons or menus instead of server-side checks, and overly broad roles that grant more access than a job function requires.

1.2.4.3 Accountability

Accountability means that the system can provide trustworthy evidence of who performed a given action and when, so that events can be reconstructed and individuals cannot plausibly deny their own actions.

> **EXAMPLE**
> A cloud administrator deletes a production database at 2 a.m. If the platform's audit log records only that "a delete operation occurred," without capturing which authenticated identity performed it, from which IP address, and at what time, the organization cannot determine who was responsible or whether the action was even authorized. Accountability failures make incident response and any subsequent legal or disciplinary action nearly impossible, even when confidentiality, integrity, and availability were never directly attacked.

Typical failures: missing or incomplete audit logs, logs that lack timestamps or user identifiers, logs stored on a system an attacker could compromise, and no protection against log tampering or deletion.

Table 1.2 summarizes the six security objectives, the core question each one answers, and the mechanisms that primarily support it.

Table 1.2 *Security objectives and the mechanisms that primarily support them*

Objective / property	Core question	Representative mechanisms
Confidentiality	Can an unauthorized party read this information?	Encryption, access control, data classification
Integrity	Can this information or behavior be changed without authorization?	Input validation, digital signatures, audit trails
Availability	Can legitimate users reach the application when they need it?	Rate limiting, redundancy, capacity planning
Authentication	Who are you?	Passwords, multi-factor authentication, certificates
Authorization	What are you allowed to do?	Role-based access control, policy engines
Accountability	Can we prove who did what, and when?	Audit logging, timestamps, correlation identifiers

1.3 Application Security Administration

Technical security controls alone do not create an effective application security program. Organizations also require appropriate people, processes, and technology: three interconnected pillars that together turn individual security practices into a repeatable program (Table 1.3).

Table 1.3 *The three pillars of application security administration*

Pillar	Role in the program	Representative examples
People	Establish responsibility and make security decisions	Developers, security engineers, security architects, Security Advocate, application owners
Process	Establish how security activities are performed consistently	Threat modeling, code review, vulnerability management, patch management, incident response
Technology	Provide automated and technical detection mechanisms	SAST, DAST, SCA, secrets scanning, runtime monitoring, CI/CD security controls

1.4 Application Security Models

Several complementary models describe how organizations structure their application security efforts in practice (Table 1.4). These models are not mutually exclusive; mature organizations typically blend elements of several.

Table 1.4 *Application security models*

Model	Core idea	Representative controls
Zero-Trust Security	Eliminate implicit trust; access is continuously evaluated rather than granted because a request came from inside the network	Strong identity verification, service-to-service authentication, signed tokens, just-in-time privileges
Shift-Left Security	Move security activities earlier in the life cycle so feedback arrives while the code is still fresh	SAST in the IDE and pull request, dependency analysis, secrets scanning, automated pre-merge testing
Defense in Depth	Use multiple independent layers so no single control failure causes complete compromise	Input validation + parameterized queries + least-privilege database accounts + authorization + monitoring
DevSecOps	Integrate security into existing development and operations workflows rather than a separate final stage	Security findings tracked in the same issue tracker and CI/CD pipeline as other engineering work
Risk-Based Security	Prioritize remediation using business context rather than technical severity alone	Scoring by asset criticality, data sensitivity, exploitability, and runtime exposure

Note. A vulnerability's technical severity alone often does not provide enough information to determine remediation priority; risk-based security supplies the missing business context.

1.5 Security Frameworks and Standards

Frameworks provide structure and common terminology for application security programs, so that organizations do not need to invent controls from scratch. Table 1.5 lists the frameworks and standards most commonly used.

Table 1.5 *Application security frameworks and standards*

Framework	Publisher	Primary focus
OWASP (Top 10 / ASVS)	Open Worldwide Application Security Project	Community-maintained catalog of common risks (Top 10) and a detailed verification standard (ASVS)
NIST SSDF (SP 800-218)	U.S. National Institute of Standards and Technology	Secure practices organized into four groups: Prepare, Protect, Produce, and Respond
ISO/IEC 27034	International Organization for Standardization	Application security management integrated into an organization's broader information security program
CIS Controls	Center for Internet Security	Prioritized, prescriptive safeguards spanning asset management, access control, and configuration
PCI DSS	PCI Security Standards Council	Mandatory controls for any system that stores, processes, or transmits payment card data

Note. The appropriate framework depends on the organization's architecture, business requirements, regulatory environment, and security objectives; most mature programs draw on more than one.

1.6 Threat Modeling

Threat modeling is a systematic security-analysis process used to identify, understand, prioritize, and mitigate threats to an information system before and throughout its implementation. Its fundamental purpose is to shift security analysis from a purely reactive activity, discovering vulnerabilities after a system has been developed, to a proactive activity in which potential attacks and their consequences are considered during system design.

*A **vulnerability** describes a weakness in a system; a **threat** describes a potential action or event that could exploit a weakness or otherwise cause harm; and **risk** reflects the potential consequences of that threat in the context of the system*. Threat modeling connects these concepts by examining how a system operates, what it is intended to protect, who interacts with it, which components communicate, where information is stored and transmitted, and where trust relationships change.

A threat-modeling process can be organized around three fundamental questions:

1. **What are we building?**
   Identify the system's components, users, assets, data flows, external dependencies, and trust boundaries.
2. **What can go wrong?**
   Systematically identify threats that could compromise confidentiality, integrity, availability, authentication, authorization, or accountability.

3. **What are we going to do about it?**
   Select appropriate preventive, detective, and corrective security controls and determine how residual risk will be managed.

These questions establish threat modeling as an iterative engineering activity rather than a one-time documentation exercise.

1.6.1 Data Flow Diagrams

A **Data Flow Diagram (DFD)** provides a structured representation of how information moves through an application or system. In security-oriented threat modeling, the DFD is not merely a functional diagram. It is an analytical model that exposes the components, interactions, data stores, and trust relationships that may become relevant to an attack. A security-oriented DFD normally represents four major types of element:

1. **External entities:** users, external applications, organizations, devices, or third-party services that interact with the system.
2. **Processes:** application components or services that receive, transform, validate, or generate information.
3. **Data stores:** databases, files, caches, local storage, message queues, backups, or other locations where information is retained.
4. **Data flows:** communications or exchanges of information between entities, processes, and data stores.

A fifth concept, the **trust boundary**, is particularly important for security analysis and is discussed next.

1.6.2 Trust Boundaries

A trust boundary is a logical boundary across which the level of trust, privilege, control, or security assurance changes. It does not necessarily correspond to a physical network boundary. A trust boundary may exist between an Internet user and an internal server, between a mobile application and a backend API, between two microservices with different privileges, or between an application and a third-party service. Whenever information crosses a trust boundary, the receiving component should not automatically assume that the sender or the transmitted information is trustworthy.

For example, consider a mobile banking application:

Mobile User → Mobile Application → Authentication API → Banking API → Database

The mobile device should generally be treated as an untrusted environment from the perspective of the backend. An attacker who controls or compromises the device may be able to inspect application files, manipulate local data, alter requests, replay communications, or instrument application execution. Consequently, security-critical decisions must not depend exclusively on controls implemented within the client application. A mobile application may provide a user interface and perform client-side validation, but the server must independently authenticate users, authorize operations, validate input, and enforce security policies.

At every trust boundary, the threat analyst should ask:

- Who can communicate with the receiving component?
- What information crosses the boundary, and is the communication authenticated?
- Is the information protected in transit?
- Can the sender be trusted to provide accurate information?
- Can the transmitted information be modified or replayed?
- Does the receiving component independently validate the information?
- What happens if the communicating component becomes unavailable?
- What privileges are granted after the information crosses the boundary?

These questions transform a functional architecture diagram into a security-oriented representation of the system.

1.6.3 Threat Identification Using STRIDE

Once the DFD has been constructed, the analyst can systematically examine its components and data flows for potential threats. One of the most widely used approaches for this purpose is **STRIDE**, a threat-modeling framework developed at Microsoft. STRIDE organizes threats into six categories, each corresponding to a different security property that may be violated (Table 1.7).

Table 1.7 *The STRIDE threat categories with definitions, examples, and typical controls*

STRIDE category	Definition	Security property violated	Examples	Typical controls
Spoofing	An attacker pretends to be another user, device, service, or system component.	Authentication / authenticity	Using stolen usernames and passwords; using a stolen authentication token; impersonating an internal service; presenting a fraudulent identity to an API.	Multi-factor authentication; secure credential management; secure session management; token validation; device or service authentication.
Tampering	Unauthorized modification of data, requests, configurations, or software, whether stored or in transit.	Integrity	Changing a transaction amount in a request (amount=100 to amount=10000); altering stored records or configurations; changing data in transit between client and server.	Authenticated and encrypted communication; server-side input validation; digital signatures; access controls; database integrity constraints.
Repudiation	An individual or system denies having performed an action, and the system lacks trustworthy evidence of what happened.	Accountability / non-repudiation	A customer disputes a transaction they initiated; an administrator denies a configuration change; an action cannot be traced to an authenticated identity.	Secure audit logging; reliable timestamps; transaction identifiers; recording the authenticated identity; tamper-resistant logs.
Information Disclosure	Sensitive information becomes accessible to an unauthorized individual, process, or system.	Confidentiality	Exposure of customer records; insecure local storage of authentication tokens; excessive detail in error messages; sensitive data in logs; improperly protected backups.	Encryption in transit and at rest; access control; least privilege; secure secrets management; data minimization; appropriate error handling.

STRIDE category	Definition	Security property violated	Examples	Typical controls
Denial of Service	An attacker or failure condition prevents legitimate users from accessing a service or significantly degrades its performance.	Availability	Flooding a login endpoint with requests; exhausting server memory or CPU; overwhelming a database connection pool; sending excessively large requests.	Rate limiting; resource quotas; request-size limits; load balancing; DDoS protection; capacity planning; monitoring and alerting.
Elevation of Privilege	An attacker obtains permissions beyond those legitimately assigned to them.	Authorization	Changing an identifier to access another user's data (/account?id=1001 to /account?id=1002); invoking administrative functions as a normal user; bypassing role checks that exist only in the client.	Server-side authorization checks; role- or attribute-based access control; least privilege; separation of duties; object-level authorization.

1.6.4 Threat Prioritization and Risk Ranking

A threat model may identify many threats, but security teams rarely have the resources to address all of them at once, so the threats must be prioritized. A practical and transparent approach is to rate each threat on two separate dimensions: **likelihood**, meaning how easy or probable it is for the threat to occur, and **impact**, meaning how serious the consequences would be if it did. Both are rated against consistent criteria, for which the simple three-level scale in Table 1.8 can be used.

Table 1.8 *Three attack levels for likelihood and impact*

Level	Likelihood	Impact
High (3)	Requires little specialized skill or access; readily available tools or techniques exist	Could affect many users, cause significant financial loss, expose sensitive information, or create safety consequences
Medium (2)	Requires some skill, privileged access, or specific conditions	Could affect an individual or limited group, or cause moderate financial or reputational damage
Low (1)	Requires advanced skill, rare access, or multiple conditions to align	Causes minor, temporary, or easily reversible harm

> SCORING FORMULA
>
> **Risk Score = Likelihood × Impact**
>
> The score ranges from 1 to 9. A higher score generally indicates that a threat should be addressed sooner.

Table 1.9 shows the resulting scores for every combination of likelihood and impact.

Table 1.9 *Risk score matrix*

Likelihood / Impact	Low (1)	Medium (2)	High (3)
High (3)	**3**	**6**	**9**
Medium (2)	**2**	**4**	**6**

Likelihood / Impact	Low (1)	Medium (2)	High (3)
Low (1)	1	2	3

A common error in threat assessment is to confuse the two dimensions. A compromise of an entire customer database has very high impact, but that does not make it highly likely; likelihood depends on the conditions needed to carry out the attack. Likewise, a technically sophisticated attack should not automatically outrank a simple one, since an attack that needs little skill but can be repeated against thousands of users may be the greater practical risk. Good practice also avoids rating everything as High and requires a written reason for each rating.

1.7 Worked Example: QuickBite Threat Model

This section applies the techniques introduced in Section 1.6 to a realistic system. The scenario is presented first, followed by 3 tasks and a model solution to each.

1.7.1 Scenario

You are the security engineer for **QuickBite**, a food delivery app. The system works as follows:

- **Customers** use a mobile app to browse restaurants, place orders, and pay by card.
- **Restaurants** use a web dashboard to accept orders and update menus and prices.
- **Drivers** use a driver app that shows pickup and drop-off addresses and shares live GPS location.
- A **backend API** connects everyone, with a database of user profiles, addresses, and order history.
- A **third-party payment provider** handles card payments.
- A **promo code system** gives discounts and refunds.

1.7.2 Tasks

1. **Draw a data flow diagram.** Show all actors, components, data stores, and data flows. Mark at least three trust boundaries.
2. **Apply STRIDE.** Identify at least two realistic threats per STRIDE category (12 or more in total). Each threat must name the affected component or data flow.
3. **Rank the threats.** Rate each one High, Medium, or Low for likelihood and impact, calculate the score (Likelihood × Impact), and select the top five using the scale in Section 1.6.4.

1.7.3 Solution to Task 1: Data Flow Diagram

Figure 1.1 shows the QuickBite system with its three trust boundaries and seven numbered data flows.

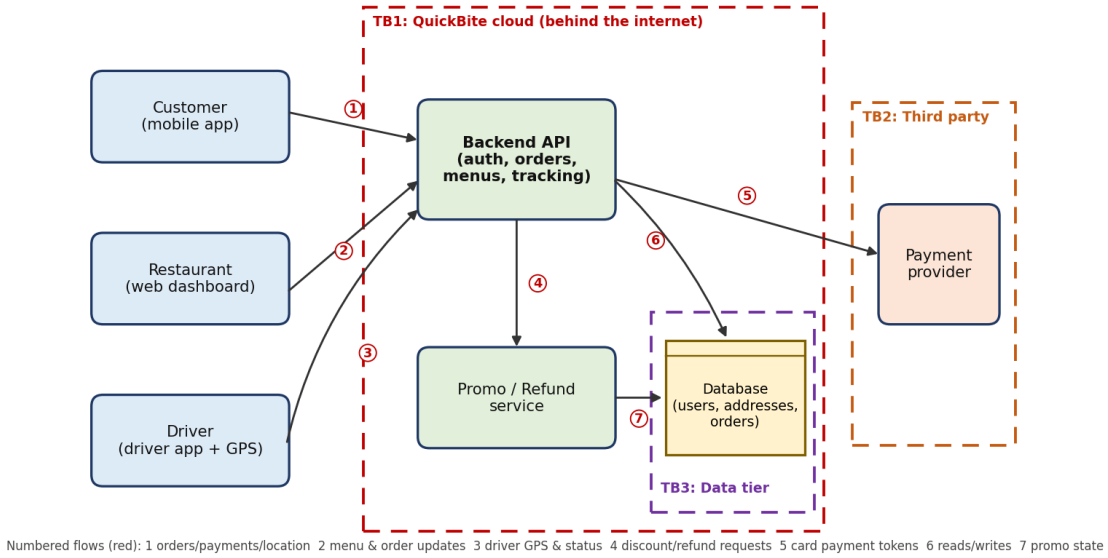


Figure 1.1 *QuickBite data flow diagram with three trust boundaries*

The three trust boundaries are:

- **TB1, internet to QuickBite cloud:** all three client devices (customer, restaurant, driver) are outside the company's control, so everything crossing this line is untrusted input.
- **TB2, QuickBite to payment provider:** a different organization with its own security. Messages must be authenticated in both directions.
- **TB3, application tier to data tier:** the database holds the most sensitive data and should accept requests only from known services with minimal rights.

The driver's GPS stream (flow 3) is shown explicitly because it is easy to forget, yet it is both personal data and a business-critical signal.

1.7.4 Solution to Task 2: STRIDE Threat Register

Each STRIDE category has at least two threats, and each names the component or flow affected (Table 1.10).

Table 1.10 *QuickBite STRIDE threat register*

ID	Category	Component / flow	Threat
S1	Spoofing	Flow 1 / Customer login / Backend API	Attacker uses credential stuffing or a stolen session token to log in as a customer, view saved addresses, and place orders on their saved card.
S2	Spoofing	Flows 2 and 3 / Restaurant and driver onboarding	A fake restaurant or fake driver account is registered to collect orders, harvest customer addresses or phone numbers, or divert payouts.

ID	Category	Component / flow	Threat
T1	Tampering	Flow 1 / order request	Customer edits the request to change item price, delivery fee, or discount (e.g. `price=12.00` becomes `price=0.10`) and the server trusts the values.
T2	Tampering	Flow 3 / driver GPS and status	A rooted driver phone sends spoofed GPS coordinates or a forged "delivered" status to collect pay without delivering.
T3	Tampering	Flow 5 / Payment	A tampered client marks an order as "paid" without a verified payment confirmation from the provider.
R1	Repudiation	Flows 1 and 3 / Order delivery	Customer claims "my order never arrived" while the driver claims it was delivered. No proof of delivery (photo, PIN, signed event) exists.
R2	Repudiation	Flow 4 / Promo / refund service	A support agent or restaurant user issues refunds or changes menu prices, then denies it. Admin actions are not logged with identity and time.
I1	Information disclosure	Flow 3 / driver app	The driver app receives the customer's full profile (phone, email, history) and keeps it after delivery, so a malicious driver can misuse it.
I2	Information disclosure	Flow 1 / Backend API: order history	IDOR flaw: changing `/orders/5521` to `/orders/5522` returns another customer's address, items, and phone number.
I3	Information disclosure	Flows 6 and 7 / Database: backups	An unencrypted database backup or over-broad DB account leaks addresses and order history if the storage is exposed.
D1	Denial of service	Flow 2 / Restaurant dashboard	Attacker or competitor floods a restaurant with fake orders so real customers are blocked and the restaurant must close.
D2	Denial of service	Flows 1 and 4 / Backend API: login and promo endpoint	A bot floods the API (or brute-forces promo codes) at dinner peak, exhausting capacity so real users cannot order.
E1	Elevation of privilege	Flow 1 / Backend API: function-level access	A customer calls restaurant-only endpoints (edit menu) because the server checks login but not role.
E2	Elevation of privilege	Flow 4 / Promo / refund service	A driver or customer reaches the internal refund endpoint, or a support agent grants themselves admin rights, and issues unlimited refunds.

1.7.5 Solution to Task 3: Risk Ranking

The scoring rule is **Score = Likelihood × Impact**, where High = 3, Medium = 2, and Low = 1 (maximum 9). When scores are equal, the tie-break order is as follows: the threat that puts personal safety at risk ranks higher, then the one that affects more users, then the one that costs more money. Affects more users means the number of people who would suffer real harm, such as loss of money, privacy, or account access, if the attack succeeded. Harm to the business alone

does not count at this step; it is covered by the cost criterion. The ranked register is given in Table 1.11.

Table 1.11 *QuickBite threats ranked by risk score*

ID	Threat	Likelihood	Impact	Score	Priority rank
I2	IDOR leaks other customers' data	High	High	**9**	1
S1	Account takeover (credential stuffing)	High	High	**9**	2
T1	Price / discount tampering	High	High	**9**	3
E1	Customer calls restaurant/admin endpoints	Medium	High	**6**	4
D2	API / promo flooding at peak time	High	Medium	**6**	5
T3	Order marked paid without verified payment	Medium	High	**6**	6
E2	Refund service abused	Medium	High	**6**	7
S2	Fake restaurant / driver accounts	Medium	Medium	**4**	8
I1	Driver keeps customer data	Medium	Medium	**4**	9
D1	Fake-order flooding of a restaurant	Medium	Medium	**4**	10
T2	Spoofed GPS / forged delivery status	Medium	Medium	**4**	11
R1	No proof of delivery	Medium	Medium	**4**	12
I3	Database or backup exposure	Low	High	**3**	13
R2	Admin actions not logged	Low	Medium	**2**	14

The **top five** threats are **I2, S1, T1, E1, and D2**.

Why this order. I2 ranks first among the three threats scoring 9 because it is the only one treated as a personal-safety risk: an attacker with an ordinary account of their own can change an order ID and collect the home addresses and phone numbers of thousands of other customers, without needing anyone else's login. S1 and T1 tie on personal safety, because the main harm of S1 is financial: the attacker spends from the victim's saved card and sees only that one account's details, not a bulk collection of other customers' addresses. The second criterion therefore decides between them. S1 ranks second because every hijacked account has a real victim who loses control of their account and saved card, and whose saved address is visible to the attacker. T1 ranks third because its harm falls mainly on the business, so it ranks below S1 at the users-affected step. Among the threats scoring 6, E1 and D2 harm customers and restaurant owners across the platform, so they rank above T3 and E2 at the users-affected step. E1 and D2 tie on personal safety and users affected, so cost decides: E1 ranks above D2 because an attacker who can edit the menu causes lasting financial damage, whereas a flood causes a temporary outage. T3 and E2 tie on personal safety, because neither exposes personal data, and on users affected, because both harm mainly the business. The cost criterion does not clearly separate them, so T3 is placed first by judgment, because a payment bypass loses the full value of every order it is applied to.

1.8 Secure Coding Practices

Secure coding means writing software so that attackers cannot misuse it, by avoiding the common mistakes that create vulnerabilities. This section presents twelve practices. Each has a short definition and an example, with a "bad versus good" comparison where it helps.

1.8.1 Input Validation

> **DEFINITION**
> Never trust data from users, files, or APIs. Check that it has the expected type, length, format, and range before using it.

```
# Bad: accepts anything
age = request.form["age"]

# Good: validate type and range
age = int(request.form["age"])
if not (0 < age < 120):
    raise ValueError("Invalid age")
```

Prefer **allow-lists** (accept only known-good values) over block-lists (reject known-bad values).

1.8.2 Parameterized Queries (Preventing SQL Injection)

> **DEFINITION**
> Keep SQL code separate from user data, so that input can never change the meaning of the query.

```
# Bad: attacker enters ' OR '1'='1
query = f"SELECT * FROM users WHERE name = '{name}'"

# Good: placeholder treats input as data only
cursor.execute("SELECT * FROM users WHERE name = %s", (name,))
```

1.10.3 Output Encoding (Preventing XSS)

> **DEFINITION**
> Convert special characters in user-provided data before displaying it, so that the browser shows it as text instead of running it as code.

```
import html

# Input: <script>stealCookies()</script>
safe = html.escape(user_comment)
# Output shown: &lt;script&gt;stealCookies()&lt;/script&gt;
```

1.8.4 Secure Password Storage

> **DEFINITION**
> Never store passwords in plain text. Store a salted hash made with a slow algorithm such as bcrypt or Argon2.

```
import bcrypt

hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
bcrypt.checkpw(password.encode(), hashed)  # verify at login
```

Never use MD5 or SHA-1 for passwords.

1.10.5 Authentication and Session Management

> **DEFINITION**
> Reliably verify who the user is, and protect their login session afterwards.

Examples:

- Use multi-factor authentication (MFA).
- Lock or slow down accounts after repeated failed logins.
- Use random session IDs, set cookies with `HttpOnly`, `Secure`, and `SameSite`, and expire sessions after inactivity.

1.8.6 Authorization and Least Privilege

> **DEFINITION**
> Check what a user is allowed to do on every request, and give users and programs only the minimum permissions they need.

```
# Bad: any logged-in user can view any invoice
invoice = get_invoice(invoice_id)

# Good: verify ownership
invoice = get_invoice(invoice_id)
if invoice.owner_id != current_user.id:
    abort(403)
```

As an example of least privilege, a web application's database account should not have permission to `DROP TABLE`.

1.8.7 Secrets Management

> **DEFINITION**
> Never hard-code passwords, API keys, or tokens in source code, and never commit them to Git.

```
# Bad
API_KEY = "sk-12345-secret"

# Good: load from environment or a secrets manager
import os
API_KEY = os.environ["API_KEY"]
```

1.8.8 Secure Error Handling and Logging

DEFINITION

Show users generic error messages, and record the details in protected logs. Never log sensitive data.

```
# Bad: leaks internals to the user
return str(exception)  # "Table 'users' not found at db.py line 42"

# Good
logger.error("DB error", exc_info=True)  # detailed, internal only
return "Something went wrong. Please try again."
```

1.8.9 Encryption in Transit and at Rest

DEFINITION

Protect data while it travels (HTTPS/TLS) and while it is stored (disk or database encryption).

Examples: force HTTPS everywhere, encrypt sensitive database fields, and use well-tested libraries. **Never invent your own encryption algorithm.**

1.8.10 Secure Defaults and Fail-Safe Design

DEFINITION

The system should be secure out of the box, and when something fails, it should fail closed (deny access) instead of open.

```
# Bad: if the check crashes, access is granted
try:
    allowed = check_permission(user)
except Exception:
    allowed = True

# Good: fail closed
try:
    allowed = check_permission(user)
except Exception:
    allowed = False
```

1.8.11 Code Review and Security Testing

DEFINITION

Find vulnerabilities before attackers do.

Examples: peer code reviews, static analysis (SAST) tools such as Bandit or SonarQube, dynamic testing (DAST), and penetration testing.

1.8.12 CSRF Protection

> DEFINITION
> **Cross-Site Request Forgery (CSRF)** is an attack that tricks a logged-in user's browser into sending an unwanted request to a website they trust. The browser automatically attaches the user's cookies, so the server thinks the request is genuine.

A simple analogy: imagine your bank honors any signed cheque that arrives with your stamp on it. An attacker cannot forge your stamp, but they can trick you into carrying their cheque to the bank yourself.